Sistemas distribuidos
(75.74) Curso Roca

Trabajo Práctico
Coordinación

23 de abril de 2026

Alejo Sendra
107716

1. Introducción

El presente trabajo implementa una posible solución al sistema de control de stock de una verdulería que cuenta con el siguiente sistema distribuido:

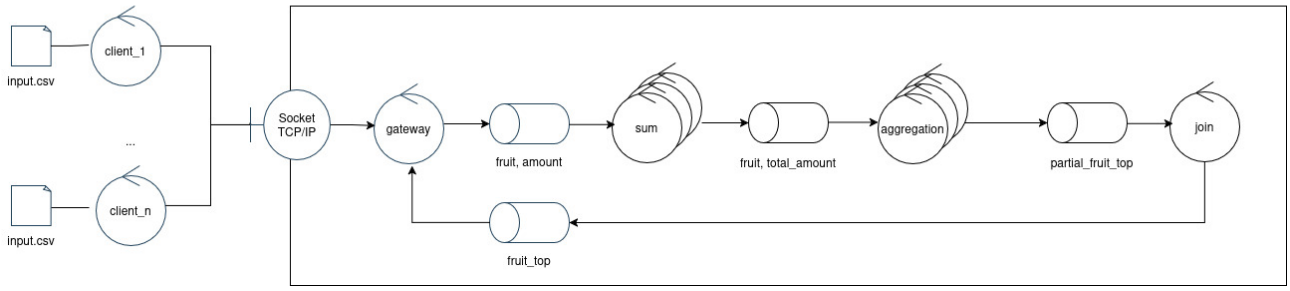

Figura 1. Diagrama de Robustez del sistema de la verduleria

Donde cada nodo cumple las siguientes funciones:

- **Client:** Lee un archivo de entrada y envía por TCP/IP pares (fruta, cantidad) al sistema. Cuando finaliza el envío de datos, aguarda un top de pares (fruta, cantidad) y vuelca el resultado en un archivo de salida csv. El criterio y tamaño del top dependen de la configuración del sistema. Por defecto se trata de un top 3 de frutas de acuerdo a la cantidad total almacenada.
- **Gateway:** Es el punto de entrada y salida del sistema. Intercambia mensajes con los clientes y las colas internas utilizando distintos protocolos.
- **Sum:** Recibe pares (fruta, cantidad) y aplica la función Suma de la clase FruitItem. Por defecto esa suma es la canónica para los números enteros, ej:

  (”manzana”, 5) + (”manzana”, 8) = (”manzana”, 13)

  Cuando se detecta el final de la ingesta de datos envía los pares (fruta, cantidad) totales a los Aggregators.
- **Aggregator:** Consolida los datos de las distintas instancias de Sum. Cuando se detecta el final de la ingesta, se calcula un top parcial y se envía esa información al Joiner.
- **Joiner:** Recibe tops parciales de las instancias del Aggregator. Cuando se detecta el final de la ingesta, se envía el top final hacia el gateway para ser entregado al cliente.

2. Coordinación entre instancias

2.1. Coordinación entre Sum's

Dado el modelo de working queue usado para distribuir las tareas entre las distintas copias de Sum, un problema importante es el de determinar el momento en el cual se debe enviar la información, de aquel cliente que termino de procesar y enviar su archivo, a la siguiente instancia (la de aggregation).

2.1.1. EOF

Para resolver esta problemática, se implementó un esquema de broadcast mediante un Exchange de RabbitMQ. Cuando una instancia de Sum detecta el mensaje de control EOF proveniente de un cliente, publica una notificación al resto de las réplicas. Esto sincroniza la descarga de buffers locales, asegurando que toda la información parcial sea enviada a la siguiente etapa.

2.1.2. Comunicación con la proxima etapa de procesamiento

Resuelto el problema de çuando enviar"la siguiente pregunta es .ª quien enviar?", aqui se puede plantear dos respuestas diferentes donde solo una ellas aprovecharia la ventaja de tener copias del nodo de Aggregation.

Para aprovechar la existencia de multiples nodos es imperativo que la misma instancia de Aggregation reciba los mismos registros (cliente,item) dado que:

- Si cualquier copia de Aggregation obtuviera cualquier registro, la instancia de aggregation solo funcionaria como una instancia de Sum.
- Si una sola instancia recibiera todos los registros de un determinado cliente, es muy probable que el computo demandado sea muy desigual entre instancias, puediendo tener copias osciosas y muchos mensajes sin procesar. Esto se debe a que algunos clientes podrian tener pocos registros y otros podrian tener millones.

Para garantizar la distribución eficiente del trabajo entre los diferentes nodos de Aggregation, se implementó un particionamiento determinístico. Utilizando un hash no criptográfico sobre la clave (cliente, item) y aplicando aritmética de módulo N (siendo N la cantidad de nodos de Aggregation), asegurando que todos los registros de una misma entidad colisionen con la misma instancia de destino. Posteriormente, al envio de todos los registros del cliente, el mensaje de EOF del cliente es enviado por todos los nodos de Sum a todos los nodos de Aggregation para asi corroborar que efectivamente todos los registros, del cliente en cuestión, han sido enviados.

2.2. Coordinación entre Aggregations

Mencionadas las problematicas anteriores y las soluciones desarrolladas, ¿ahora es necesario sincronizar a los diferentes nodos de Aggregation? la respuesta seria que no, dado que cada nodo de Sum envia la cantidad almacenda correspondiente al registro (cliente. item), previo a enviar el EOF y que la instancia de Aggregation no envia ni calcula el top de items hasta que todos los nodos de Sum le hayan enviado el EOF. Los diferentes nodos de Aggregation ya se encuentran sicronizados dada la forma de operar de los nodos de Sum.

3. Aprovechamiento de las copias

3.1. Sum

Dado que las diferentes copias del nodo obtienen sus unidades de trabajo de la working queue compartida, y que los unicos mensajes replicados entre las multiples copias son los mensajes de EOF. Es evidente que a mayor cantidad de copias, del nodo, mayor sera la velocidad de procesamiento y esta estara acotada por la capacidad del middleware de entregar los mensajes a los diferentes nodos.

3.2. Aggregation

Gracias a que las diferentes copias de Sum son deterministicas en cuanto a que nodo de Aggregation derivar los registro según (cliente,item), no solo permite distribuir equitativamente la carga de trabajo sino que permite aprovechar la capacidad de los multiples nodos para calcular el top parcial y desechar información que ya no es necesaria. Esto se debe a que las cantidades para un determinado item y cliente se encuentran concentradas en el mismo nodo, de forma que un item que no se encuentra en el top parcial no va a poder estar en el top global dado que no existen otros registros de ese item para ese cliente.

4. Aclaraciones adicionales

4.1. Justificación de multithreading

El diseño del nodo Sum utiliza un modelo multi-threading para gestionar la lectura de los multiples canales de comunicación. Si bien el GIL de python impide la ejecución en paralelo de múltiples hilos en diferentes núcleos de CPU, La misma documentación indica que la perdida de performance es minima con respecto a escenarios de multiprocesamiento en lo que refiere a operaciones limitadas a entrada/salida. Debido a que las tareas del nodo, donde se emplea multithreading, son predominantemente de entrada/salida. Entonces, debido al menor costo de recursos del multithreading, y a la ventaja de compartir un espacio de direccionamiento común. Esto facilita el acceso sincronizado a las estructuras de datos compartidas sin la complejidad y latencia adicional que introducirían mecanismos IPC.